

DS-551 Final Project:

Deep Reinforcement Learning: Indoor Navigation in 3D Environments

Objective

The objective of this project is to design and train a Deep reinforcement learning (DRL) agent that can navigate in a partially observable 3D environment. Students will apply and compare DRL algorithms such as Deep Q-Networks (DQN), Proximal Policy Optimization (PPO), or their variants.

Project Overview

In this project, students will use the “FourRooms” environment from [MiniWorld](#), a lightweight 3D simulation platform for reinforcement learning research. The agent starts in one of four connected rooms and must reach a red box placed in a different room.

The task is challenging due to Partial observability of the agent (first-person vision only). The reward structure is as follows:

$$r = 1 - 0.2 \frac{\text{step count}}{\text{max episode steps}}$$

The agent gets zero reward on every normal step. It only receives reward once it gets close enough to the red box. The max episode steps is 250. If the agent reaches the red box immediately/very early the reward is close to 1. If it reaches the goal at step 250, the reward is 0.8. and if never reaches the goal before the episode ends then the total reward is 0.

Key Tasks:

1. Installation & Setup

- Install and run the MiniWorld FourRooms environment and Familiarize yourself with the environments.
- Visualize agent observations and actions
- Understand the reward structure and episode termination conditions

2. Training RL Agents

- Implement at least one DRL algorithm (such as DQN, Double DQN, or PPO, ...)
- Train the agent to achieve maximum reward.
- Compare performance by plotting **rewards vs. timesteps**
- Generate a **simulation video** of the trained model in action.

You can explore the following items to make your agent perform satisfactorily:

- hyperparameter tuning (e.g. batch size, replay buffer size, ϵ -greedy ...)
- Network architecture design (e.g. use of convolutional neural networks (CNN))
- uniform replay buffer versus prioritized replay buffer.
- frame stacking or recurrent memory (to handle partial observability). See the remark below in the Bonus section.

Bonus:

Implement Recurrent policies (e.g., [DRQN](#), Recurrent PPO); Recurrent policies are important because they let the agent retain memory of past observations, which is crucial in partially observable environments where a single observation doesn't fully describe the state. Another common approach is frame stacking, which approximates this by feeding several recent observations at once, but recurrent models are more flexible since they can learn what information to remember and for how long.

You may:

- Use established DRL libraries (e.g., [Stable-Baselines3](#), [RLlib](#), [CleanRL](#)).
- Or implement your own PPO/DQN from scratch (for extra credit).

Team Collaboration

- Teams of **two students** are allowed.

Deliverables

1. 10-Minute Pre-recorded Presentation
 - Cover the architecture, implementation challenges, and results.
 - Include simulation videos of the trained model and final plots.
2. Code Submission
 - Well-documented, organized, and reproducible code.
3. Presentation Materials
 - Slides (PDF/PPT)